

Alineamiento Local

Alumno: Josue Samuel Philco Puma

30 de abril de 2026

1. Introducción

El manejo de secuencias biológicas implica su almacenamiento, procesamiento y análisis mediante herramientas computacionales. Estas suelen representarse en formatos estándar como **FASTA** permitiendo su manipulación eficiente en distintos programas. Y mediante estos análisis, es posible la identificación de regiones funcionales, comparar organismos, estudiar mutaciones y comprender procesos evolutivos.

2. Ejercicios

2.1. Algoritmo Smith-Waterman

Para realizar este laboratorio debemos implementar el algoritmo **Smith-Waterman** que es para alineamiento local, este algoritmo como tal va a permitir identificar regiones de alta similitud entre dos secuencias biológicas, para ello vamos a aplicar programación dinámica para su implementación.

1. Debemos recibir como entrada dos secuencias y junto con ello sus parámetros de puntuación (**match**, **mismatch** y **gap**),
2. Para realizar este proceso vamos a tener una matriz de tamaño $(n + 1) * (m + 1)$ donde n y m van a ser las longitudes de las secuencias. Toda la matriz se inicializa con ceros, lo que permite realizar alineamiento local.
3. Una vez tengamos la matriz, debemos recorrerla calculando 3 posibles valores: valor de la diagonal (representa el match o mismatch), el valor de arriba (representa a un gap en la segunda secuencia) y el valor de la izquierda (representa gap de la primera secuencia).
4. Para cada celda se selecciona el máximo entre cero y los tres valores calculados. El uso de cero es fundamental porque permite reiniciar el alineamiento cuando los puntajes se vuelven negativos. Y cada vez que se calcula el valor se debe comparar con el puntaje máximo global.
5. Al completar el proceso, se debe realizar un **traceback** en la posición del puntaje máximo. Durante ese proceso se compara de donde ha provenido el valor.
6. Al terminar en un valor cero obtendremos las secuencias alineadas, junto a ello también el puntaje máximo y la información adicional.

Ahora que tenemos los pasos vamos a proceder con el código del algoritmo, pero además de solo el código los parámetros necesarios para poder realizar el proceso del algoritmo

1. **Valores en el puntaje:** Para este usaremos los valores siguientes: **match=+2**, **mismatch=-1** y **gap=-2**.

```
1 const int MATCH_SCORE = 2;  
2 const int MISMATCH_PENALTY = -1;  
3 const int GAP_PENALTY = -2;
```

2. **Construcción de la matriz y alineamiento óptimo:** Esta parte saldrá una vez tengamos el algoritmo implementado, además como salida debemos considerar las secuencias alineadas que nos da como resultado, el path que se ha realizado y como es que ha construido la matriz.

Entonces, el algoritmo de **Smith-Water** se vería de la siguiente manera.

```
1 struct AlignmentResult {
2     std::string sequence_1;
3     std::string sequence_2;
4     int score;
5     int start_pos_seq1, end_pos_seq1;
6     int start_pos_seq2, end_pos_seq2;
7     std::vector<std::vector<int>> score_matrix;
8     std::vector<std::pair<int, int>> alignment_path;
9 };
10
11 AlignmentResult smith_waterman(const std::string &sequence_1, const std::string &
    sequence_2, int match_score, int mismatch_penalty, int gap_penalty, const std:::
    string &export_file = "") {
12     int n = (int)sequence_1.length();
13     int m = (int)sequence_2.length();
14     int max_score = 0, end_i = 0, end_j = 0;
15
16     std::vector<std::vector<int>> score_matrix(n + 1, std::vector<int>(m + 1, 0));
17
18     for (int i = 1; i <= n; i++) {
19         for (int j = 1; j <= m; j++) {
20             int diagonal_score = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
                sequence_2[j - 1] ? match_score : mismatch_penalty);
21             int up_score = score_matrix[i - 1][j] + gap_penalty;
22             int left_score = score_matrix[i][j - 1] + gap_penalty;
23             score_matrix[i][j] = std::max({0, diagonal_score, up_score, left_score
                });
24
25             if (score_matrix[i][j] > max_score) {
26                 max_score = score_matrix[i][j];
27                 end_i = i;
28                 end_j = j;
29             }
30         }
31     }
32
33     std::string aligned_seq1, aligned_seq2;
34     std::vector<std::pair<int, int>> alignment_path;
35     int i = end_i, j = end_j;
36
37     while (i > 0 && j > 0 && score_matrix[i][j] > 0) {
38         alignment_path.push_back({i, j});
39         int diagonal = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
            sequence_2[j - 1] ? match_score : mismatch_penalty);
40         int up = score_matrix[i - 1][j] + gap_penalty;
41
42         if (score_matrix[i][j] == diagonal) {
43             aligned_seq1 += sequence_1[i - 1];
44             aligned_seq2 += sequence_2[j - 1];
45             i--;
46             j--;
47         }
48         else if (score_matrix[i][j] == up) {
49             aligned_seq1 += sequence_1[i - 1];
50             aligned_seq2 += '-';
51             i--;
52         }
53         else {
54             aligned_seq1 += '-';
55             aligned_seq2 += sequence_2[j - 1];
56             j--;
57         }
58     }
```

```

59
60     std::reverse(aligned_seq1.begin(), aligned_seq1.end());
61     std::reverse(aligned_seq2.begin(), aligned_seq2.end());
62     std::reverse(alignment_path.begin(), alignment_path.end());
63
64     return {aligned_seq1, aligned_seq2, max_score, i, end_i - 1, j, end_j - 1,
65           score_matrix, alignment_path};
66 }

```

Lo que realizará son los pasos resaltados anteriormente solo que los resultados los irá guardando en un archivo .txt por separado, obteniendo los siguientes resultados:

[illegible][illegible]

(a) Secuencia 1 alineada.

(b) Secuencia 2 alineada.

Figura 1: Resultados de la secuencia alineada 1 (a) y secuencia alineada 2 (b)

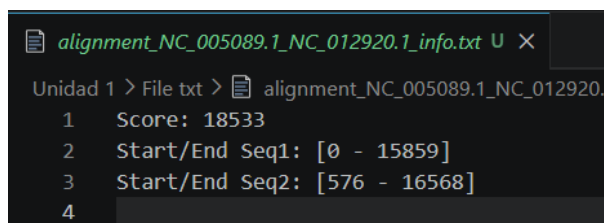


Figura 2: Puntaje máximo e información de las secuencias

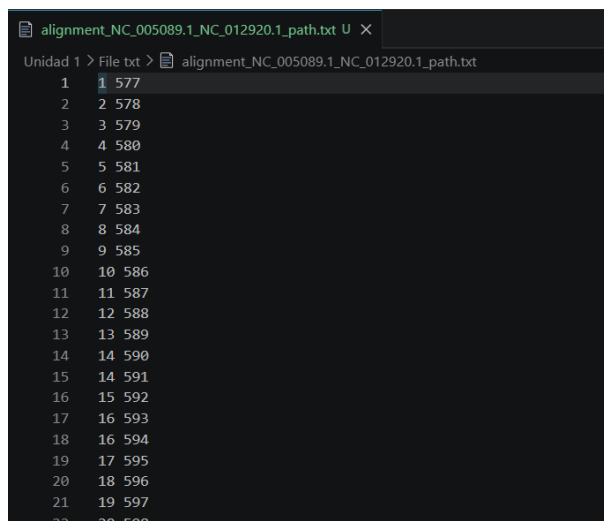


Figura 3: Path resultante.



- **Archivo de información** (`_info.txt`): Resume los resultados del alineamiento local, incluyendo el score óptimo obtenido, las posiciones de inicio y fin del alineamiento en cada secuencia, y las secuencias alineadas mostrando los gaps introducidos por el algoritmo.
- **Archivo de matriz** (`_matrix.txt`): Contiene la matriz de puntuación generada por el algoritmo Smith-Waterman. Cada valor representa el score de similitud acumulado hasta esa posición, y constituye la base para la visualización mediante el mapa de calor (*heatmap*).
- **Archivo de camino** (`_path.txt`): Contiene las coordenadas del traceback óptimo, es decir, el recorrido desde el score máximo hasta el origen del alineamiento, el cual corresponde a la línea diagonal visible en el mapa de calor.

Ahora viene una parte que puede cambiar en algo todo, y es la parte del **gap** donde nos preguntamos que pasa si tenemos un valor diferente a lo que tenemos, para ello probaremos con 3 gaps distintos:

- ```
PS D:\UNIBA ERCC\Uno semestre\Bioinformatica\Unidad 1> .\LocalAlignment.exe .\Files\WGBI_musculus.fasta .\Files\WGBI_mitochondrion.fasta
 Match score: 2
 Mismatch: -1
 Gap penalty: -1
Sequences exported to: File txt/sequences1.txt
Sequences exported to: File txt/sequences2.txt
Aligning NC_005089.1 with NC_012920.1...
Alignment matrix exported to: File txt/alignment_NC_005089.1_NC_012920.1_matrix.txt
Alignment path exported to: File txt/alignment_NC_005089.1_NC_012920.1_path.txt
Alignment info exported to: File txt/alignment_NC_005089.1_NC_012920.1_info.txt
Aligned sequences exported to: File txt/alignment_NC_005089.1_NC_012920.1_aligned.txt
```

[illegible][illegible]

Figura 6: Resultados de la secuencia alineada 1 (a) y secuencia alineada 2 (b) con gap -1







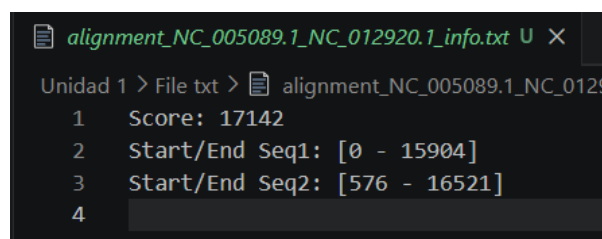
- ```

● PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1> .\LocalAlignment.exe .\Files\NCBI
    Match score: 2
    Mismatch: -1
    Gap penalty: -5
    Sequences exported to: File txt/sequences1.txt
    Sequences exported to: File txt/sequences2.txt
    Aligning NC_005089.1 with NC_012920.1...
    Alignment matrix exported to: File txt/alignment_NC_005089.1_NC_012920.1_matrix.txt
    Alignment path exported to: File txt/alignment_NC_005089.1_NC_012920.1_path.txt
    Alignment info exported to: File txt/alignment_NC_005089.1_NC_012920.1_info.txt
    Aligned sequences exported to: File txt/alignment_NC_005089.1_NC_012920.1_aligned.txt

```

[illegible][illegible]

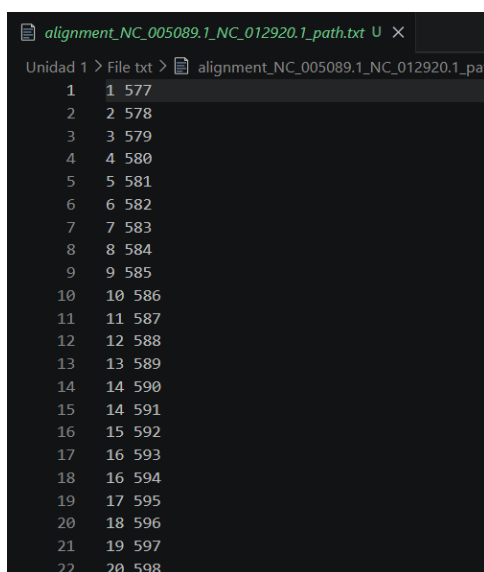
Figura 16: Resultados de la secuencia alineada 1 (a) y secuencia alineada 2 (b) con gap -5.



```

1 alignment_NC_0050891_NC_012920_1_matrix.txt U X
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1
```

7



```

alignment_NC_005089.1_NC_012920.1_path.txt U X
Unidad 1 > File txt > alignment_NC_005089.1_NC_012920.1_pat
1 1 577
2 2 578
3 3 579
4 4 580
5 5 581
6 6 582
7 7 583
8 8 584
9 9 585
10 10 586
11 11 587
12 12 588
13 13 589
14 14 590
15 14 591
16 15 592
17 16 593
18 16 594
19 17 595
20 18 596
21 19 597
22 20 598
  
```

Figura 19: Path con gap -5.

Entonces, la diferencia de 3027 puntos entre gap -1 y gap -5 refleja cuánto del alineamiento depende de gaps para mantenerse. En secuencias mitocondriales de especies distintas como *Mus musculus* y humano, es esperable que existan inserciones y deleciones evolutivas, por lo que un gap penalty intermedio como -2 o -3 captura mejor la similitud real entre ambas.

2.3. Medición del tiempo con secuencias largas

Ahora vamos a medir el tiempo de ejecución que demora en realizar todo el proceso de alineamiento local con **Smith-Water**, este tiempo lo tenemos que medir en el loop donde esta la función quedando de esta manera:

```

1 for (const auto &record1 : records1) {
2     for (const auto &record2 : records2) {
3         std::cout << "Aligning " << record1.identifier << " with " << record2.
4             identifier << "..." << std::endl;
5
6         auto start = std::chrono::high_resolution_clock::now();
7         AlignmentResult result = smith_waterman(record1.sequence, record2.sequence,
8             MATCH_SCORE, MISMATCH_PENALTY, GAP_PENALTY);
9         auto end = std::chrono::high_resolution_clock::now();
10        std::chrono::duration<double> elapsed = end - start;
11        std::cout << "Time: " << std::fixed << std::setprecision(3) << elapsed.
12            count() << " seconds\n\n";
13
14        std::string base_path = "File txt/alignment_" + record1.identifier + "_" +
15            record2.identifier;
16        export_alignment(result, base_path);
17        export_aligned_sequences(result, base_path + "_aligned.txt");
18    }
19 }
  
```

Entonces si ejecutamos nos saldrá un tiempo total de **38.882 segundos** para estas secuencias mitocondriales. Si probamos con archivos con muchas más secuencias sería un tiempo mucho mayor en realizar todo el proceso de **Smith-Water**.


```
PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1> .\LocalAlignment.exe .\Files\WABI_musculus.fasta .\Files\WABI_mitochondrion.fasta
Match score: 2
Mismatch: -1
Gap penalty: -2
Sequences exported to: File txt/sequences1.txt
Sequences exported to: File txt/sequences2.txt
Aligning NC_005089.1 with NC_012920.1...
Time: 38.882 seconds

Alignment matrix exported to: File txt/alignment_NC_005089.1_NC_012920.1.matrix.txt
Alignment path exported to: File txt/alignment_NC_005089.1_NC_012920.1.path.txt
Alignment info exported to: File txt/alignment_NC_005089.1_NC_012920.1.info.txt
Aligned sequences exported to: File txt/alignment_NC_005089.1_NC_012920.1_aligned.txt
PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1>
```

Figura 20: Tiempo calculado en el algoritmo Smith-Water.

2.4. Generar heatmap mediante la matriz obtenida

Ahora, para finalizar con los datos obtenidos de las secuencias y las matrices, podemos visualizar esta matriz mediante un **heatmap**, para ello usaremos Python para la visualización.

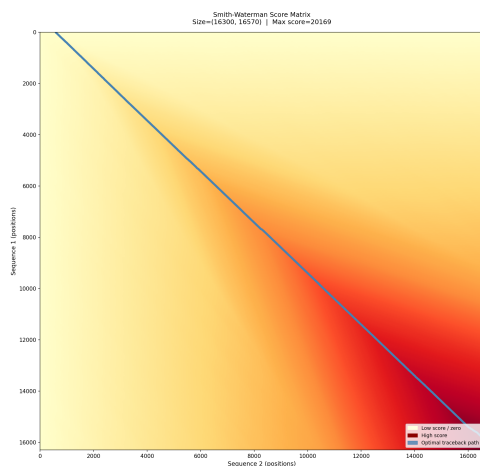
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.patches as mpatches
4 import sys
5 import os
6
7 def load_matrix(matrix_path):
8     return np.loadtxt(matrix_path, dtype=int)
9
10 def load_path(path_path):
11     path = set()
12     with open(path_path, 'r') as f:
13         for line in f:
14             parts = line.split()
15             if len(parts) == 2:
16                 path.add((int(parts[0]), int(parts[1])))
17     return path
18
19 def plot_heatmap(matrix_path, path_path, output_img=None):
20     print(f"Reading matrix: {matrix_path}")
21     print(f"Reading path: {path_path}")
22
23     H = load_matrix(matrix_path)
24     path = load_path(path_path)
25
26     print(f"Matrix size: {H.shape}")
27     print(f"Path points: {len(path)}")
28     print(f"Max score: {H.max()}")
29
30     if output_img is None:
31         base_name = os.path.splitext(os.path.basename(matrix_path))[0]
32         output_img = os.path.join("Heatmaps", base_name + ".png")
33
34     fig, ax = plt.subplots(figsize=(14, 12))
35
36     im = ax.imshow(H, cmap="YlOrRd", aspect="auto", interpolation="nearest")
37     plt.colorbar(im, ax=ax, label="Score", fraction=0.03, pad=0.04)
38
39     if path:
40         path_coords = [(j, i) for (i, j) in path if i < H.shape[0] and j < H.shape[1]]
41         if path_coords:
42             xs, ys = zip(*path_coords)
43             ax.scatter(xs, ys, s=10, c='steelblue', alpha=0.8, linewidths=0)
44
45     ax.set_xlabel("Sequence 2 (positions)", fontsize=11)
46     ax.set_ylabel("Sequence 1 (positions)", fontsize=11)
47     ax.set_title("Heatmap of Sequence 1 vs Sequence 2 Alignment Scores")
```

```

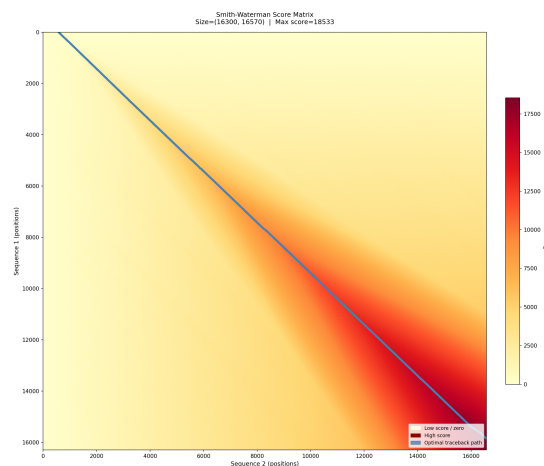
48     f"Smith-Waterman Score Matrix\n"
49     f"Size={H.shape} | Max score={H.max()} ",
50     fontsize=12, pad=15
51 )
52
53     patch_path = mpatches.Patch(color='steelblue', alpha=0.8, label='Optimal
54         traceback path')
55     patch_high = mpatches.Patch(color='darkred', label='High score')
56     patch_low = mpatches.Patch(color='lightyellow', label='Low score / zero')
57     ax.legend(handles=[patch_low, patch_high, patch_path], loc='lower right',
58         fontsize=9)
59
60     plt.tight_layout()
61     plt.savefig(output_img, dpi=150, bbox_inches='tight')
62     print(f"Saved: {output_img}")
63     plt.close()

```

Entonces con eso vamos a poder visualizar como se distribuyó la matriz, obteniendo las siguientes figuras:



(a) Heatmap matriz gap -1.



(b) Heatmap matriz gap -2.

Figura 21: Heatmap para gap -1 (a) y Heatmap para gap -2 (b)



(a) Heatmap matriz gap -3.



(b) Heatmap matriz gap -5.

Figura 22: Heatmap para gap -3 (a) y Heatmap para gap -5 (b)

3. Programa completo

Este trabajo fue realizado en C++ (algoritmo Smith-Water) y en Python (visualización de heatmaps), en el programa de C++ hay funciones extras que ayudan a dividir la información requerida en archivos .txt.

3.1. Código C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <fstream>
5 #include <algorithm>
6 #include <iomanip>
7 #include <chrono>
8
9 const int MATCH_SCORE = 2;
10 const int MISMATCH_PENALTY = -1;
11 const int GAP_PENALTY = -2;
12 const int DISPLAY = 100;
13
14 struct FastaRecordStructure {
15     std::string identifier;
16     std::string sequence;
17 };
18
19 struct AlignmentResult {
20     std::string sequence_1;
21     std::string sequence_2;
22     int score;
23     int start_pos_seq1, end_pos_seq1;
24     int start_pos_seq2, end_pos_seq2;
25     std::vector<std::vector<int>> score_matrix;
26     std::vector<std::pair<int, int>> alignment_path;
27 };
28
29 void read_file_fasta(const std::string &file_fasta, std::vector<
    FastaRecordStructure> &records) {
30     std::ifstream file(file_fasta);
31     if (!file.is_open()) {
32         std::cerr << "Error opening file: " << file_fasta << std::endl;
33         return;
34     }
35
36     auto strip_cr = [](std::string &line) {
37         if (!line.empty() && line.back() == '\r') {
38             line.pop_back();
39         }
40     };
41
42     std::string line, sequence, header;
43     while (std::getline(file, line)) {
44         strip_cr(line);
45         if (line.empty()) {
46             continue;
47         }
48
49         if (line[0] == '>') {
50             if (!header.empty()) {
51                 records.push_back({header, sequence});
52                 sequence.clear();
53             }
54             std::string raw_header = line.substr(1);
```

```

55     size_t first_space = raw_header.find(' ');
56     header = (first_space != std::string::npos) ? raw_header.substr(0,
        first_space) : raw_header;
57 }
58 else {
59     for (char &c : line) {
60         c = std::toupper((unsigned char)c);
61     }
62     sequence += line;
63 }
64 }
65
66 if (!header.empty()) {
67     records.push_back({header, sequence});
68 }
69
70 file.close();
71 }
72
73 void export_sequences(const std::string &output_path, const std::vector<
    FastaRecordStructure> &records) {
74     std::ofstream output(output_path);
75     if (!output.is_open()) {
76         std::cerr << "Error opening file for writing: " << output_path << std::endl
            ;
77         return;
78     }
79
80     for (const auto &record : records) {
81         output << ">" << record.identifier << "\n";
82         for (size_t i = 0; i < record.sequence.length(); i += DISPLAY) {
83             output << record.sequence.substr(i, DISPLAY) << "\n";
84         }
85     }
86
87     output.close();
88     std::cout << "Sequences exported to: " << output_path << std::endl;
89 }
90
91 void export_aligned_sequences(const AlignmentResult &result, const std::string &
    output_path) {
92     std::ofstream output(output_path);
93     if (!output.is_open()) {
94         std::cerr << "Error opening file for writing: " << output_path << std::endl
            ;
95         return;
96     }
97
98     output << "Secuencia 1:\n";
99     for (size_t k = 0; k < result.sequence_1.length(); k += DISPLAY)
100         output << result.sequence_1.substr(k, DISPLAY) << "\n";
101
102     output << "\nSecuencia 2:\n";
103     for (size_t k = 0; k < result.sequence_2.length(); k += DISPLAY)
104         output << result.sequence_2.substr(k, DISPLAY) << "\n";
105
106     output.close();
107     std::cout << "Aligned sequences exported to: " << output_path << std::endl;
108 }
109
110 AlignmentResult smith_waterman(const std::string &sequence_1, const std::string &
    sequence_2, int match_score, int mismatch_penalty, int gap_penalty, const std:::
    string &export_file = "") {

```

```

111     int n = (int)sequence_1.length();
112     int m = (int)sequence_2.length();
113     int max_score = 0, end_i = 0, end_j = 0;
114
115     std::vector<std::vector<int>> score_matrix(n + 1, std::vector<int>(m + 1, 0));
116
117     for (int i = 1; i <= n; i++) {
118         for (int j = 1; j <= m; j++) {
119             int diagonal_score = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
120                 sequence_2[j - 1] ? match_score : mismatch_penalty);
121             int up_score = score_matrix[i - 1][j] + gap_penalty;
122             int left_score = score_matrix[i][j - 1] + gap_penalty;
123             score_matrix[i][j] = std::max({0, diagonal_score, up_score, left_score
124                 });
125
126             if (score_matrix[i][j] > max_score) {
127                 max_score = score_matrix[i][j];
128                 end_i = i;
129                 end_j = j;
130             }
131         }
132     }
133
134     std::string aligned_seq1, aligned_seq2;
135     std::vector<std::pair<int, int>> alignment_path;
136     int i = end_i, j = end_j;
137
138     while (i > 0 && j > 0 && score_matrix[i][j] > 0) {
139         alignment_path.push_back({i, j});
140         int diagonal = score_matrix[i - 1][j - 1] + (sequence_1[i - 1] ==
141             sequence_2[j - 1] ? match_score : mismatch_penalty);
142         int up = score_matrix[i - 1][j] + gap_penalty;
143
144         if (score_matrix[i][j] == diagonal) {
145             aligned_seq1 += sequence_1[i - 1];
146             aligned_seq2 += sequence_2[j - 1];
147             i--;
148             j--;
149         }
150         else if (score_matrix[i][j] == up) {
151             aligned_seq1 += sequence_1[i - 1];
152             aligned_seq2 += '-';
153             i--;
154         }
155         else {
156             aligned_seq1 += '-';
157             aligned_seq2 += sequence_2[j - 1];
158             j--;
159         }
160     }
161
162     std::reverse(aligned_seq1.begin(), aligned_seq1.end());
163     std::reverse(aligned_seq2.begin(), aligned_seq2.end());
164     std::reverse(alignment_path.begin(), alignment_path.end());
165
166     return {aligned_seq1, aligned_seq2, max_score, i, end_i - 1, j, end_j - 1,
167         score_matrix, alignment_path};
168 }
169
170 void export_alignment(const AlignmentResult &result, const std::string &base_path)
171 {
172     int n = (int)result.score_matrix.size() - 1;
173     int m = (int)result.score_matrix[0].size() - 1;

```

```

169
170 {
171     std::ofstream output(base_path + "_matrix.txt");
172     if (output.is_open()) {
173         for (int r = 0; r <= n; r++) {
174             for (int c = 0; c <= m; c++) {
175                 output << result.score_matrix[r][c];
176                 if (c < m) {
177                     output << " ";
178                 }
179             }
180             output << "\n";
181         }
182         output.close();
183         std::cout << "Alignment matrix exported to: " << base_path + "_matrix.
184             txt" << std::endl;
185     }
186     else {
187         std::cerr << "Error opening file for writing: " << base_path + "_matrix
188             .txt" << std::endl;
189     }
190 }
191 {
192     std::ofstream output(base_path + "_path.txt");
193     if (output.is_open()) {
194         for (const auto &pos : result.alignment_path) {
195             output << pos.first << " " << pos.second << "\n";
196         }
197         output.close();
198         std::cout << "Alignment path exported to: " << base_path + "_path.txt"
199             << std::endl;
200     }
201     else {
202         std::cerr << "Error opening file for writing: " << base_path + "_path.
203             txt" << std::endl;
204     }
205 }
206 {
207     std::ofstream output(base_path + "_info.txt");
208     if (output.is_open()) {
209         output << "Score: " << result.score << "\n";
210         output << "Start/End Seq1: [" << result.start_pos_seq1 << " - " <<
211             result.end_pos_seq1 << "]\n";
212         output << "Start/End Seq2: [" << result.start_pos_seq2 << " - " <<
213             result.end_pos_seq2 << "]\n";
214         output.close();
215         std::cout << "Alignment info exported to: " << base_path + "_info.txt"
216             << std::endl;
217     }
218     else {
219         std::cerr << "Error opening file for writing: " << base_path + "_info.
220             txt" << std::endl;
221     }
222 }
223 }
224
225 int main(int argc, char *argv[]) {
226     if (argc != 3) {
227         std::cerr << "Usage: " << argv[0] << " <file1.fasta> <file2.fasta>" << std
228             ::endl;
229         return 1;

```



```

223     }
224
225     std::cout << "   Match score: " << MATCH_SCORE << "\n";
226     std::cout << "   Mismatch: " << MISMATCH_PENALTY << "\n";
227     std::cout << "   Gap penalty: " << GAP_PENALTY << "\n";
228
229     std::vector<FastaRecordStructure> records1, records2;
230     read_file_fasta(argv[1], records1);
231     read_file_fasta(argv[2], records2);
232
233     export_sequences("File txt/sequences1.txt", records1);
234     export_sequences("File txt/sequences2.txt", records2);
235
236     for (const auto &record1 : records1) {
237         for (const auto &record2 : records2) {
238             std::cout << "Aligning " << record1.identifier << " with " << record2.
                identifier << "..." << std::endl;
239
240             auto start = std::chrono::high_resolution_clock::now();
241             AlignmentResult result = smith_waterman(record1.sequence, record2.
                sequence, MATCH_SCORE, MISMATCH_PENALTY, GAP_PENALTY);
242             auto end = std::chrono::high_resolution_clock::now();
243             std::chrono::duration<double> elapsed = end - start;
244             std::cout << "Time: " << std::fixed << std::setprecision(3) << elapsed.
                count() << " seconds\n\n";
245
246             std::string base_path = "File txt/alignment_" + record1.identifier + "_"
                + record2.identifier;
247             export_alignment(result, base_path);
248             export_aligned_sequences(result, base_path + "_aligned.txt");
249         }
250     }
251
252     return 0;
253 }

```

3.2. Código Python

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.patches as mpatches
4  import sys
5  import os
6
7  def load_matrix(matrix_path):
8      return np.loadtxt(matrix_path, dtype=int)
9
10 def load_path(path_path):
11     path = set()
12     with open(path_path, 'r') as f:
13         for line in f:
14             parts = line.split()
15             if len(parts) == 2:
16                 path.add((int(parts[0]), int(parts[1])))
17     return path
18
19 def plot_heatmap(matrix_path, path_path, output_img=None):
20     print(f"Reading matrix: {matrix_path}")
21     print(f"Reading path: {path_path}")
22
23     H = load_matrix(matrix_path)
24     path = load_path(path_path)

```

```

25
26 print(f"Matrix size: {H.shape}")
27 print(f"Path points: {len(path)}")
28 print(f"Max score: {H.max()}")
29
30 if output_img is None:
31     base_name = os.path.splitext(os.path.basename(matrix_path))[0]
32     output_img = os.path.join("Heatmaps", base_name + ".png")
33
34 fig, ax = plt.subplots(figsize=(14, 12))
35
36 im = ax.imshow(H, cmap="YlOrRd", aspect="auto", interpolation="nearest")
37 plt.colorbar(im, ax=ax, label="Score", fraction=0.03, pad=0.04)
38
39 if path:
40     path_coords = [(j, i) for (i, j) in path if i < H.shape[0] and j < H.shape
41                      [1]]
42     if path_coords:
43         xs, ys = zip(*path_coords)
44         ax.scatter(xs, ys, s=10, c='steelblue', alpha=0.8, linewidths=0)
45
46 ax.set_xlabel("Sequence 2 (positions)", fontsize=11)
47 ax.set_ylabel("Sequence 1 (positions)", fontsize=11)
48 ax.set_title(
49     f"Smith-Waterman Score Matrix\n"
50     f"Size={H.shape} | Max score={H.max()}",
51     fontsize=12, pad=15
52 )
53
54 patch_path = mpatches.Patch(color='steelblue', alpha=0.8, label='Optimal
55     traceback path')
56 patch_high = mpatches.Patch(color='darkred', label='High score')
57 patch_low = mpatches.Patch(color='lightyellow', label='Low score / zero')
58 ax.legend(handles=[patch_low, patch_high, patch_path], loc='lower right',
59     fontsize=9)
60
61 plt.tight_layout()
62 plt.savefig(output_img, dpi=150, bbox_inches='tight')
63 print(f"Saved: {output_img}")
64 plt.close()
65
66 if __name__ == "__main__":
67     if len(sys.argv) < 3:
68         print("Usage: python ShowMatrix.py <_matrix.txt> <_path.txt> [output.png]")
69         sys.exit(1)
70
71     matrix_file = sys.argv[1]
72     path_file = sys.argv[2]
73     output_file = sys.argv[3] if len(sys.argv) >= 4 else None
74
75     plot_heatmap(matrix_file, path_file, output_file)

```